Anatomia de um agente LangChain

o que o framework padroniza e o que continua sendo decisão sua

agente.py — o exemplo mínimo, numa tela só

```python
from langchain.agents import create_agent
from langchain.agents.middleware import ModelCallLimitMiddleware
from langchain_anthropic import ChatAnthropic

modelo = ChatAnthropic(model="claude-haiku-4-5", max_retries=3)

ferramentas = await client.get_tools()     # vindas do servidor MCP
ferramentas = [f for f in ferramentas if f.name in PERMITIDAS]

agente = create_agent(
    modelo,
    ferramentas,
    system_prompt=INSTRUCAO,
    middleware=[ModelCallLimitMiddleware(run_limit=25, exit_behavior="end")],
)

resposta = await agente.ainvoke({"messages": [HumanMessage(evento)]})
```

O LangChain padroniza

a parte mecânica

= as linhas lilás do código

- **conversa com o modelo** — chamada e novas tentativas
- **registro das ferramentas** — create_agent apresenta ao modelo
- **laço de execução** — modelo → ferramenta → modelo …

Decisão sua

o que muda o comportamento

= as linhas laranja do código

- **quais ferramentas** — só as PERMITIDAS, das do MCP
- **qual instrução** — INSTRUCAO: o papel do agente
- **quando parar** — no máximo 25 chamadas ao modelo

Não depende de framework nenhum

vale com LangChain, com outro ou sem nenhum

o critério

a fronteira do que o sistema pode fazer

modelo + ferramentas + instrução + limite = o agente

no código, um agente é pouca coisa

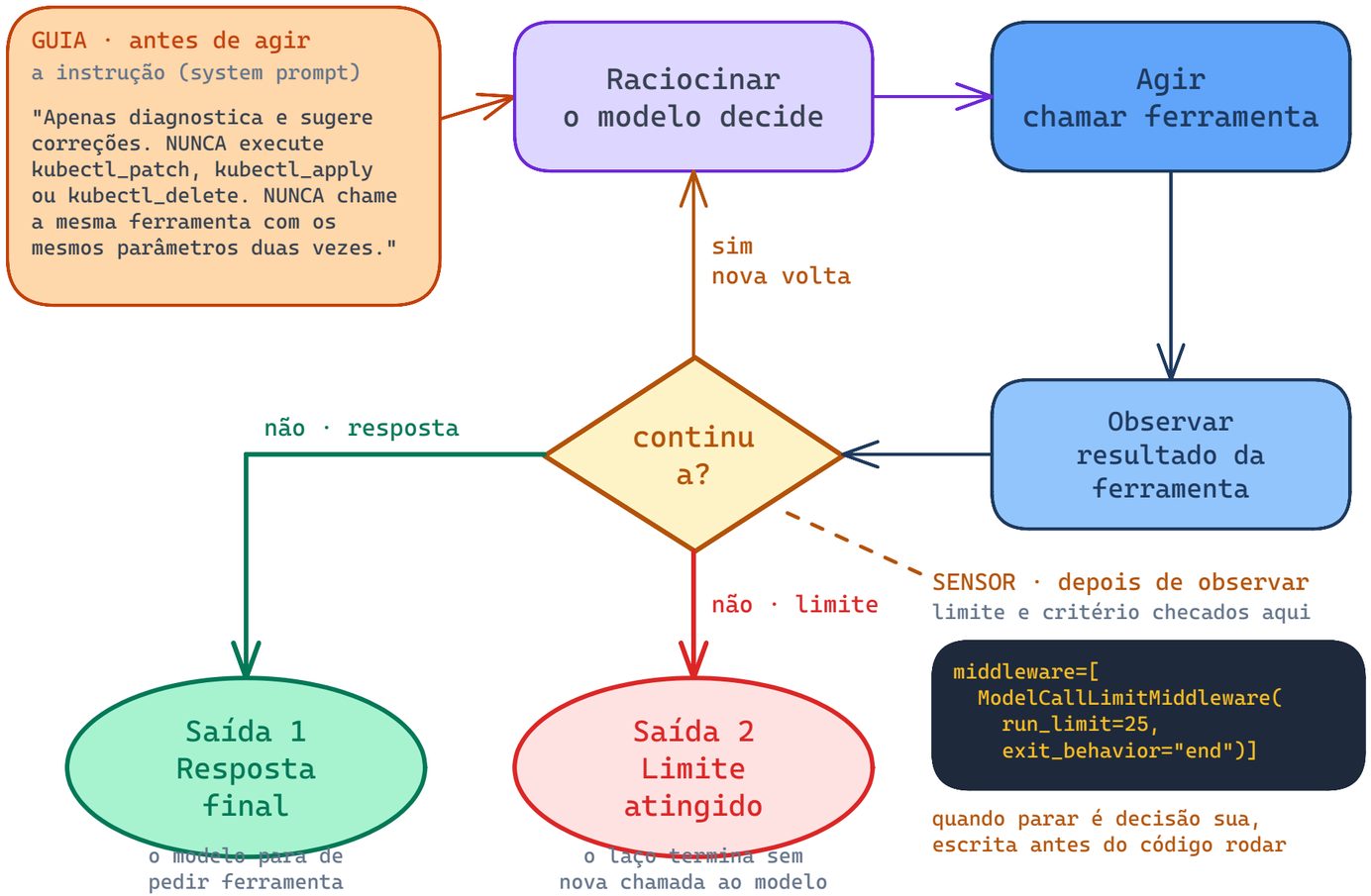
Ler e dirigir o código gerado, não escrever do zero

revisão obrigatória:

a fronteira de permissão · o critério de parada · o que toca o cluster

O laço ReAct e a condição de saída

O ciclo · guia antes, sensor depois



Rastro de execução · uma volta = 3 etapas

```
evento recebido:
Warning · OOMKilled · pod worker-relatorios-7d9f

● volta 1
[raciocinar]  o container foi encerrado por memória;
              preciso ver o limite configurado
[agir]        kubectl_describe(resourceType="pod",
                name="worker-relatorios-7d9f")
[observar]    Limits: memory 128Mi ·
              Last State: OOMKilled · Restart Count: 6

● volta 2
[raciocinar]  o limite é baixo; os logs mostram
              o que consumiu memória?
[agir]        kubectl_logs(name="worker-relatorios-7d9f",
                previous=true)
[observar]    carregando lote de 50.000 registros
              em memória ...

● volta 3 → saída 1
[raciocinar]  tenho evidência suficiente para concluir
[resposta final]  causa: limite de 128Mi abaixo
                  do consumo do lote
                  correção proposta: aumentar o limite
                  de memória do deployment
```

entre uma volta e outra, o losango "continua?" decide;
na volta 3 o modelo para de pedir ferramenta

O mesmo padrão em dois lugares

agente de codificação ⟷ ReAct ⟷ agente AIOps em Python

a ferramenta que a sala usava · o sistema que a sala constrói

Raciocinar → agir → observar → até a resposta ou o limite

A fiação com o cluster

mesmo servidor, dois transportes, dois consumidores

um servidor, a permissão muda de lado

A · Agente de codificação

Agente de codificação

transporte stdio

processo filho do agente

o servidor é registrado dentro do agente

⇒ o agente não tem a capacidade

TRAVA

no servidor variável de ambiente

MCP Server Kubernetes

um só servidor

ferramentas (todas)

kubectl_get

kubectl_describe

kubectl_logs

kubectl_apply

kubectl_patch

kubectl_delete

kubectl_scale

...

Cluster Kubernetes kind

terminal

```
$ ENABLE_UNSAFE_STREAMABLE_HTTP_TRANSPORT=1 \
  PORT=3001 npx mcp-server-kubernetes

# servidor HTTP autônomo
# sobe com todas as ferramentas
```

B · Aplicação Python (LangChain)

Aplicação Python LangChain

FILTRO

no cliente

HTTP · localhost:3001/mcp

chegam todas as ferramentas

passam só kubectl_get · kubectl_describe · kubectl_logs

Ponto de retomada

health-check do servidor antes de seguir

cliente.py

```
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
    {"k8s": {"transport": "http", "url": "http://localhost:3001/mcp"}}
)
ferramentas = await client.get_tools()


PERMITIDAS = {"kubectl_get", "kubectl_describe", "kubectl_logs"}
ferramentas = [f for f in ferramentas if f.name in PERMITIDAS]
```

o filtro mora aqui

Onde a permissão mora

● Agente de codificação

transporte stdio
registrado dentro do agente
trava no servidor, variável de ambiente

⇒ o agente não tem a capacidade

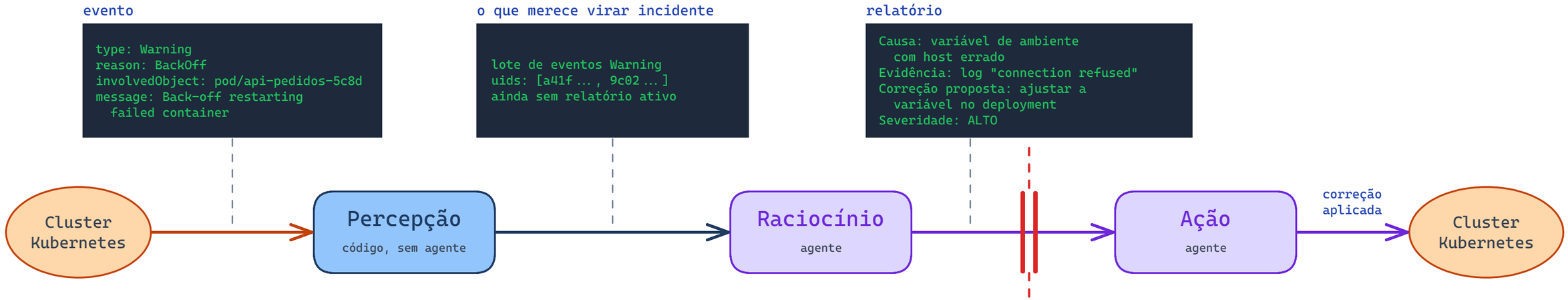
● Aplicação Python

transporte HTTP :3001/mcp
servidor autônomo
filtro no cliente

⇒ convenção escrita no código

As três engrenagens

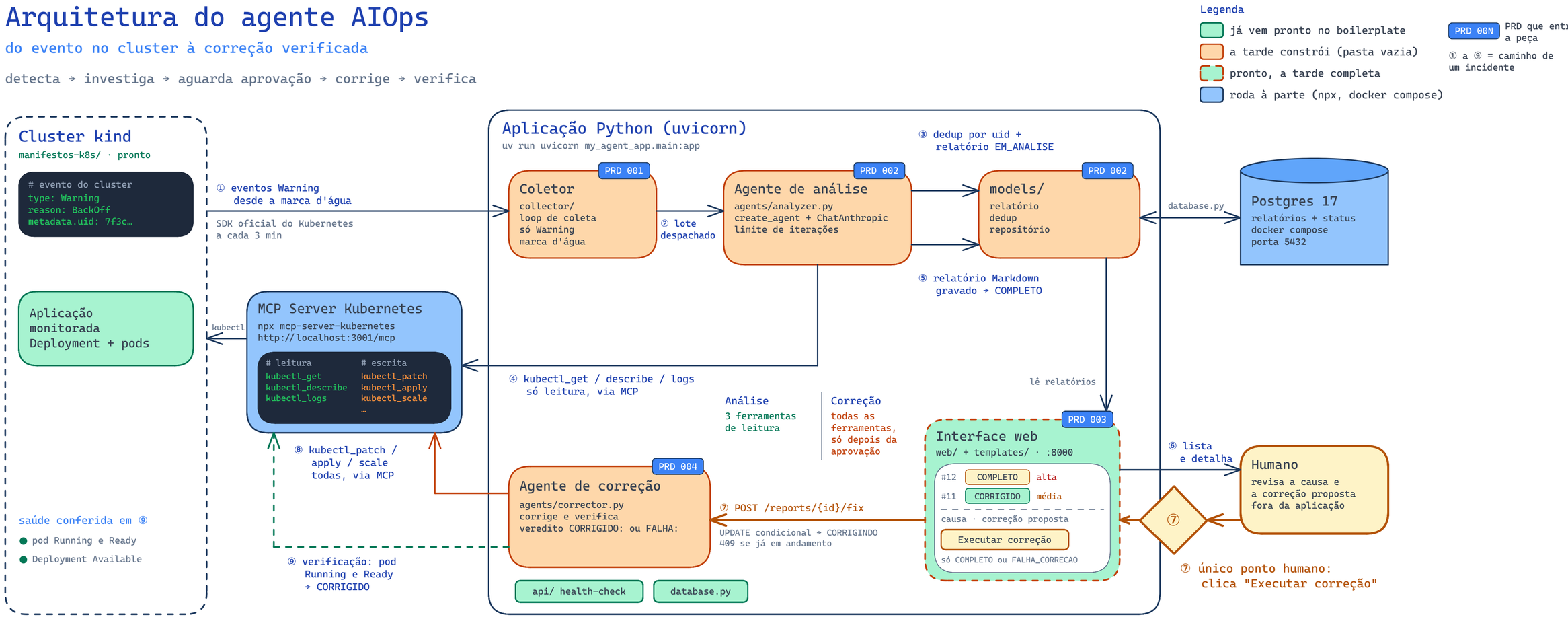
separar em engrenagens é separar responsabilidade e permissão



Arquitetura do agente AIOps

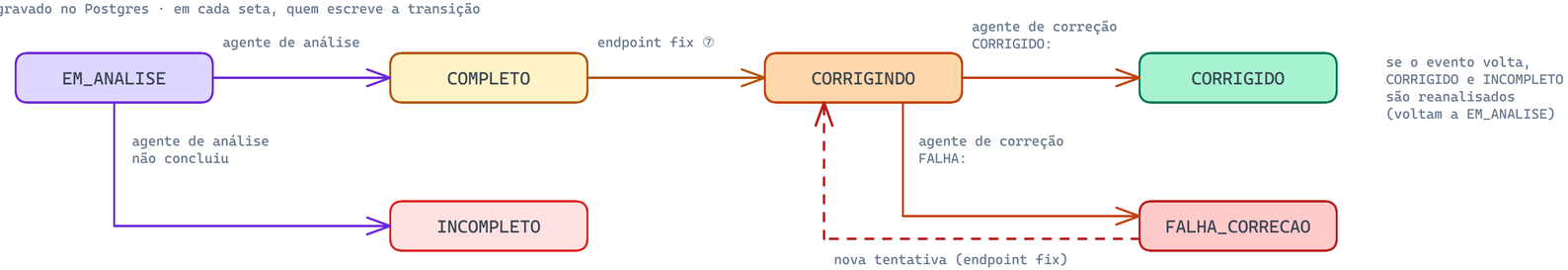
do evento no cluster à correção verificada

detecta → investiga → aguarda aprovação → corrige → verifica



Status do relatório

gravado no Postgres · em cada seta, quem escreve a transição



Onde cada engrenagem mora

src/my_agent_app/

- collector/ ← PRD 001 · vazio
- agents/ ← PRD 002 e 004 · vazio
- models/ ← PRD 002 · vazio
- web/ + templates/ ← PRD 003 · pronto, completa
- api/ health-check · pronto
- database.py pronto

docs/prds/ 001 · 002 · 003 · 004

um change do OpenSpec para cada PRD

manifestos-k8s/ · docker-compose · prontos